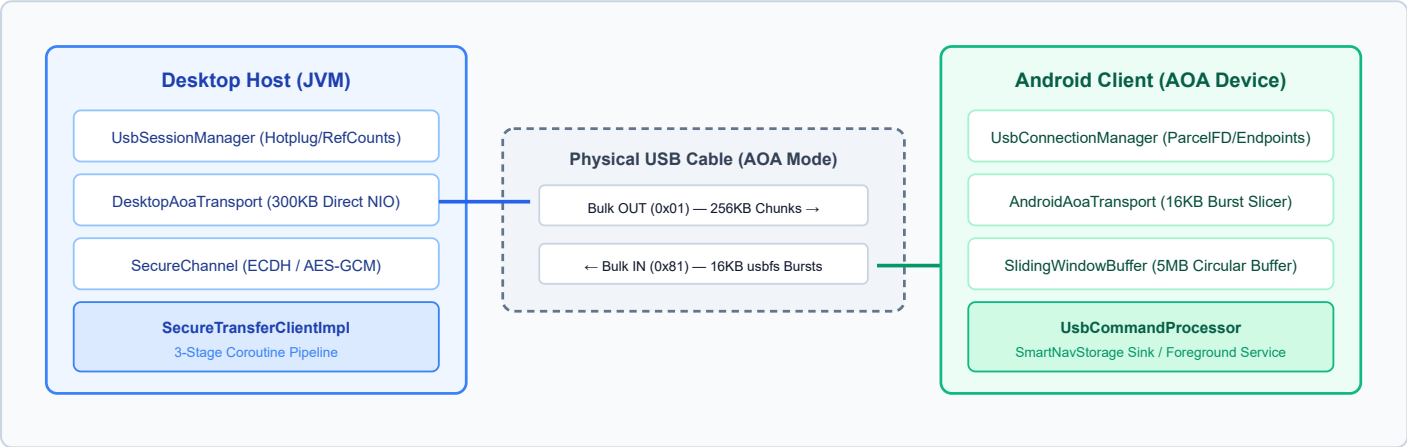
SecureQT SDK & USB Data Transfer Protocol Specification

Comprehensive architectural reference covering multiplatform SDK bindings, hardware USB communication managers, packet wire schemas, sliding window streaming, and cryptographic file pipelines.

1. Executive Architecture & Component Topology

The **SecureQT SDK** provides a unified multiplatform protocol stack connecting a Desktop Host (running JVM/Compose Desktop with native `libusb`) and an Android Client (running Android Open Accessory APIs) over high-speed physical USB bulk pipes.



The SDK entry point is `com.example.secureqt.sdk.SecureQtSdk`, exposing the foundational protocol modules:

```
val Commands = CommandConstants   // Inner encrypted RPC command byte IDs
val Packets  = PacketTypes        // 5-byte framing envelope packet type markers
val Codec    = PacketCodec        // Binary framing, length decoders, and zero-copy encoders
val Utils    = FormatUtils        // High-speed string, size, and speed formatting helpers
```

2. End-to-End Packet Structures & Binary Schemas

All USB traffic is structured in a two-tier framing protocol: outer **Transport Wire Envelopes** managed by `PacketCodec`, and inner **Encrypted Application Payloads** carrying file blocks or `CommandConstants`.

2.1 Universal Wire Header (5 Bytes)

Every frame on the USB bus begins with this fixed 5-byte sequence:

Type Byte (1B)	Payload Length N (4 Bytes, Big-Endian)
Offset 0 (uint8)	Offset 1..4 (uint32) — Total body size following header

2.2 Detailed Envelope Packet Types (`PacketTypes`)

A. `TYPE_PUBLIC_KEY` (0x01) — ECDH P-256 Public Key Handshake

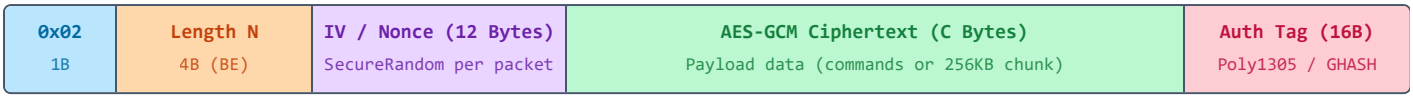
Transmitted unencrypted during initial pairing. Carries SEC1 uncompressed point format (65 bytes):

0x01	Len = 65	0x04	EC Coordinate X (32 Bytes)	EC Coordinate Y (32 Bytes)
1B	4B (BE)	SEC1 Tag	Big-Endian Scalar	Big-Endian Scalar

Total on-wire size: **70 bytes** (5 bytes header + 65 bytes payload).

B. `TYPE_DATA` (0x02) — Authenticated Encrypted Payload (AES-256-GCM)

Transports all encrypted commands, queries, responses, and 256KB file streaming blocks:



Total on-wire size for a 256KB chunk: **262,177 bytes** (5B header + 12B IV + 262,144B ciphertext + 16B tag). Total overhead = exactly 33 bytes.

C. `TYPE_EOF` (0x03) — Stream Termination Delimiter

Dispatched immediately after the final file chunk or dynamic ZIP stream to gracefully close sinks:



Total on-wire size: **5 bytes** (Zero payload).

D. `TYPE_PING` (0x04) & `TYPE_CANCEL` (0x06) — Control Packets

Packet Constant	Type Byte	Payload Len	Semantic Wire Behavior
`TYPE_PING`	`0x04`	`0 Bytes`	Heartbeat / Ready ACK signal transmitted during handshake and mode switches.
`TYPE_ERROR`	`0x05`	`N Bytes`	Carries UTF-8 error string; peer cancels pending job and triggers UI error callback.
`TYPE_CANCEL`	`0x06`	`0 Bytes`	High-priority abort frame; immediately stops active read/write jobs and purges partial files.

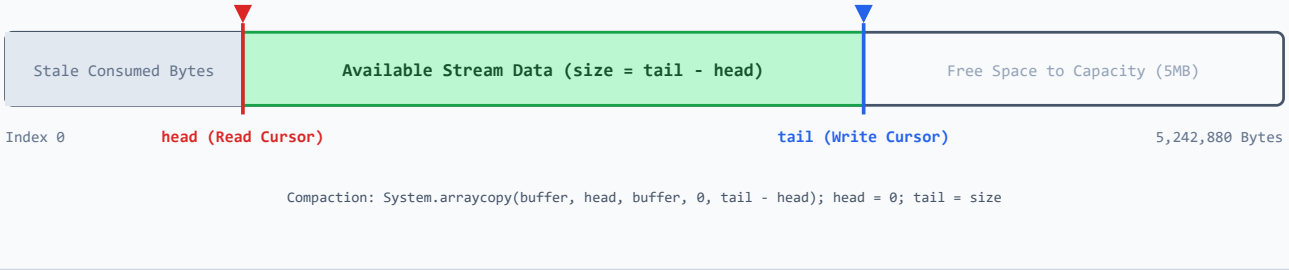
3. Application Command Packets (`CommandConstants`)

Application commands are encapsulated *inside* the decrypted plaintext of `TYPE_DATA` packets:

Command Constant	ID	Plaintext Payload Schema	Response Schema
`CMD_LIST_DIR`	`0x00`	`[0x00][4B PathLen][UTF-8 Path]`	`[4B Count M]`, then M packets: `[1B isDir][8B Size][4B NameLen][UTF-8 Name]`
`CMD_SEND_FILE_START`	`0x01`	`[0x01][4B PathLen][UTF-8 Path][8B TotalFileSize]`	Passive listener opens `FileOutputStream`, then receives 256KB `TYPE_DATA` chunks.
`CMD_FETCH_FILE_START`	`0x02`	`[0x02][4B PathLen][UTF-8 Path]`	`[8B FileSize]`, followed by continuous 256KB `TYPE_DATA` chunks until `TYPE_EOF`.
`CMD_FETCH_DIR_ZIP_START`	`0x03`	`[0x03][4B PathLen][UTF-8 Path]`	`[8B 0L]`, followed by dynamic on-the-fly ZIP stream chunks until `TYPE_EOF`.
`CMD_DISCONNECT`	`0x04`	`[0x04]` (1 Byte)	Peer resets cryptographic session, flushes buffers, and closes descriptor.
`CMD_SEND_DIR_ZIP_START`	`0x05`	`[0x05][4B PathLen][UTF-8 Path][8B 0L]`	Sender streams dynamic ZIP chunks; receiver unzips on-the-fly to target directory.
`CMD_DELETE_FILE`	`0x06`	`[0x06][4B PathLen][UTF-8 Path]`	`[1B Status]`: `0x01` = Success, `0x00` = Failed.
`CMD_RENAME_FILE`	`0x07`	`[0x07][4B OldLen][OldPath][4B NewLen][NewName]`	`[1B Status]`: `0x01` = Success, `0x00` = Failed.
`CMD_CREATE_FOLDER`	`0x08`	`[0x08][4B PathLen][UTF-8 Path]`	`[1B Status]`: `0x01` = Success, `0x00` = Failed.
`CMD_SET_DEVICE_ID`	`0x09`	`[0x09][4B IdLen][UTF-8 DeviceIdentifier]`	Registers remote device hardware ID into UI session map.
`CMD_CANCEL_TRANSFER`	`0x0F`	`[0x0F]` (1 Byte)	Application-level abort; peer stops loop and deletes incomplete target file.

4. Sliding Window Buffer (`SlidingWindowBuffer`)

Due to USB packet coalescing and fragmentation across OS kernel boundaries, reads rarely return full logical frames. `SlidingWindowBuffer` preallocates a contiguous **5 MB circular memory slab**, utilizing moving cursor pointers to eliminate memory churn and garbage collection halts:



Framing Ingestion Sequence:

1. Incoming raw USB chunks are appended via `append(data, 0, len)`, advancing `tail`.
2. `has(5)` validates header availability. `take(5)` extracts envelope bytes; `head` advances by 5.
3. Payload length N is decoded via `Codec.decodePayloadLength(header)`.
4. The loop suspends until `has(N)` is true. `take(N)` yields the payload with zero intermediary copies.
5. When `tail + incoming > buffer.size`, `compact()` shifts active bytes to index 0.

5. End-to-End File Transfer Pipeline & Line Diagram

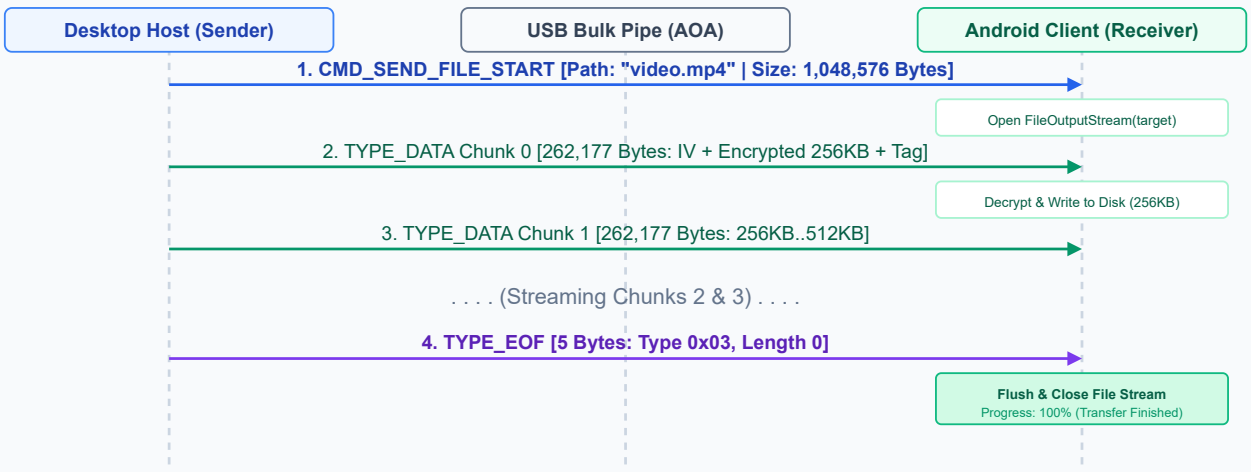
File transfer utilizes an asynchronous 3-stage coroutine pipeline decoupling disk I/O, parallel cryptography, and physical USB writing:

Producer-Consumer Channel Pipeline:

```
Disk Reader Coroutine                → Channel<ByteArray>(capacity = 32)  AES-GCM Encryptor
(Dispatchers.IO)                     →                                    (Dispatchers.Default)
```

```
AES-GCM Encryptor          → Channel<ByteArray>(capacity = 32) →          USB Bulk Writer (Dispatchers.IO)
```

5.1 Detailed File Transfer Sequence & Timing Diagram



6. Zero-Disk Streaming Directory ZIP Pipeline

Recursive folder transfers avoid creating temporary archive files on disk by utilizing inter-thread piped memory streams on both ends:

Sender Pipeline: `localDir.walkTopDown()` → `ZipOutputStream` → `PipedOutputStream / PipedInputStream` → 256KB `TYPE_DATA`

Receiver Pipeline: `TYPE_DATA` Chunks → `PipedOutputStream / PipedInputStream` → `ZipInputStream` → Extract Entries to Target Folder

7. Summary Technical Reference

Specification Parameter	Value / Limit	Subsystem Reference
Physical USB Transport	Android Open Accessory (AOA) 2.0 Bulk IN / OUT	`DesktopAoaTransport` / `AndroidAoaTransport`
Android usbfs Safe Burst	16,384 Bytes (16 KB safe physical burst)	`AndroidAoaTransport.MAX_USBFS_BURST_SIZE`
Desktop Direct NIO Buffer	307,200 Bytes (300 KB off-heap direct buffer)	`DesktopAoaTransport.readBuffer`
Sliding Window Slab	5,242,880 Bytes (5 MB preallocated circular buffer)	`SlidingWindowBuffer`
Frame Header Size	5 Bytes (1-Byte Type + 4-Byte Payload Length BE)	`PacketCodec.HEADER_SIZE`
Encryption Suite	ECDH P-256 Key Exchange + AES-256-GCM	`SecureSession` / `AesGcmEngine`
Crypto Overhead per Packet	33 Bytes (5B Header + 12B IV + 16B Poly1305 Tag)	`PacketCodec.encodeDataPacket()`
Transfer Block Size	262,144 Bytes (256 KB logical file chunks)	`SecureTransferClientImpl.CHUNK_SIZE`